Table des matières

Labo 06 OLTP — Agrégation avancée et pivots sur le transactionnel 1
Le terrain, en une minute 1
Partie 1 — Du GROUP BY au ROLLUP 2
Partie 2 — À la carte, étiqueté, croisé 3
Partie 3 — Ce que le contraste vous a montré 8
Et ensuite ? 9

Labo 06 OLTP — Agrégation avancée et pivots sur le transactionnel

Laboratoire du Cours 06, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On reprend le laboratoire OLAP sur l'autre terrain : la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Comptez une heure.

Les deux voies — Ce labo est le jumeau de `Labo_06_OLAP_SQL.md`, et il le suppose fait. C'est le cours où les deux voies divergent le plus : les extensions du `GROUP BY` sont identiques des deux côtés, mais `PIVOT` n'existe pas ici. Gardez les deux fichiers ouverts côte à côte.

Identifiants en minuscules — Le portage PostgreSQL d'AdventureWorks range les tables dans des schémas en minuscules et replie les noms de colonnes de la même façon. Une colonne de ce labo pose un problème de plus : elle s'appelle `group`, qui est un mot réservé du langage. Il faut l'écrire entre guillemets doubles, `t."group"`, sans quoi le moteur croit lire un `GROUP BY`.

Le terrain, en une minute

Table	Ce qu'elle contient	Grain
`sales.salesorderheader`	l'en-tête d'une commande : dates, territoire, totaux	une commande
`sales.salesorderdetail`	les lignes d'une commande : produit, quantité, prix	une ligne de commande
`sales.salesterritory`	les territoires de vente : nom, code pays, continent	un territoire
`person.countryregion`	le libellé des pays, à partir de leur code	un pays

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Grain
`production.product` et sa chaîne	le catalogue, sa sous-catégorie, sa catégorie	un produit

Deux différences avec l'entrepôt sautent aux yeux dès cette table. Il n'y a toujours pas de dimension temps, donc l'année se découpe à la main sur `orderdate`. Et le pays n'est pas écrit en clair : `sales.salesterritory` ne porte qu'un code sur deux lettres, `countryregioncode`, qu'il faut joindre à `person.countryregion` pour obtenir « Canada » plutôt que « CA ».

```
SELECT territoryid, name, countryregioncode, "group"
FROM sales.salesterritory
ORDER BY "group", name;                -- 10 territoires, 3 continents, 6 pays
```

C'est un exemple parfait de ce que fait une alimentation d'entrepôt : `DimSalesTerritory` porte directement `SalesTerritoryCountry` en toutes lettres, parce que quelqu'un a résolu ce code une fois pour toutes.

Partie 1 — Du `GROUP BY` au `ROLLUP`

Théorie : `Theorie_06_SQL.md`, sections 1 et 3.

D'abord le `GROUP BY` que vous connaissez, les ventes par année et par pays, un seul niveau :

```
SELECT EXTRACT(YEAR FROM h.orderdate)::int AS annee,
       c.name                              AS pays,
       SUM(d.linetotal)                    AS ventes
FROM sales.salesorderheader AS h
JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN sales.salesterritory   AS t ON t.territoryid  = h.territoryid
JOIN person.countryregion   AS c ON c.countryregioncode = t.countryregioncode
GROUP BY 1, 2
ORDER BY annee, pays;
```

Comptez les jointures : quatre tables pour une question que l'entrepôt réglait en trois. La différence n'est pas dans le calcul, elle est dans le chemin.

Maintenant le `ROLLUP`, les mêmes lignes, plus un sous-total par année, plus le total général :

```
SELECT EXTRACT(YEAR FROM h.orderdate)::int AS annee,
       c.name                              AS pays,
       SUM(d.linetotal)                    AS ventes
FROM sales.salesorderheader AS h
JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN sales.salesterritory   AS t ON t.territoryid  = h.territoryid
JOIN person.countryregion   AS c ON c.countryregioncode = t.countryregioncode
```


© 2026 iAlexMG. Tous droits réservés.

```
GROUP BY ROLLUP (EXTRACT(YEAR FROM h.orderdate)::int, c.name)
ORDER BY annee, pays;
```

Observez. Le résultat a la même forme que sur l'entrepôt, aux libellés près. `ROLLUP`, `GROUPING SETS`, `CUBE` et `GROUPING()` sont du SQL standard, présents sur PostgreSQL depuis la version 9.5 : rien à réapprendre.

Une chose mérite d'être notée. L'expression `EXTRACT(...)` est répétée en entier dans le `ROLLUP`. PostgreSQL accepterait l'alias `annee` à sa place, comme dans tout `GROUP BY` (Cours 02, section 7), mais cette tolérance est une extension, pas une règle du langage : un moteur qui suit la norme à la lettre refuse l'alias comme la position. Sur l'entrepôt, la question ne se posait pas : `CalendarYear` était une vraie colonne. Nommer le calcul une fois pour toutes règle la question sur tous les moteurs, et c'est ce qu'on fait dans la suite du labo.

Partie 2 — À la carte, étiqueté, croisé

Toutes les requêtes qui suivent lisent les mêmes cinq colonnes, au prix des mêmes quatre jointures. Plutôt que de recopier la même CTE à chaque étape, posez-la une fois pour toutes dans une vue temporaire :

```
CREATE TEMP VIEW ventes AS
SELECT EXTRACT(YEAR FROM h.orderdate)::int AS annee,
       c.name                              AS pays,
       t."group"                           AS continent,
       d.linetotal                         AS montant
FROM sales.salesorderheader AS h
JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN sales.salesterritory   AS t ON t.territoryid  = h.territoryid
JOIN person.countryregion   AS c ON c.countryregioncode = t.countryregioncode;

SELECT COUNT(*) FROM ventes;            -- 121 317 lignes
```

Trois choses à savoir sur cette vue. Elle ne stocke rien : c'est la requête qui est enregistrée, et elle se relance à chaque lecture. Le mot `TEMP` la range dans un schéma propre à votre session : elle disparaît quand vous vous déconnectez, et vous devrez la reposer à la reconnexion. Et à l'intérieur de la vue, `annee` n'est plus un alias mais une vraie colonne du résultat : le `GROUP BY` la lit sans dépendre de la tolérance d'un moteur. Une CTE rendrait le même service, au prix d'une recopie par requête ; la vue vous évite ces recopies, et c'est le seul motif de la préférer ici.

2.1 — `GROUPING SETS` et `CUBE`

Théorie : `Theorie_06_SQL.md`, sections 2 et 4.

Objectif : voir la différence entre choisir ses niveaux et tout croiser.

```
SELECT annee, pays, SUM(montant) AS ventes
FROM ventes
```


© 2026 iAlexMG. Tous droits réservés.

```sql
GROUP BY GROUPING SETS ((annee), (pays), ())
ORDER BY annee, pays;

SELECT annee, pays, SUM(montant) AS ventes
FROM ventes
GROUP BY CUBE (annee, pays)
ORDER BY annee, pays;
```

Observez. 11 lignes pour les `GROUPING SETS`, les 4 années, les 6 pays et le total général ; 35 pour le `CUBE`, qui ajoute les 24 croisements année par pays. Le Cours 05 vous a donné, sans le dire, la forme portable d'une requête du Cours 06 : nommer le calcul avant de regrouper dessus.

2.2 — Étiqueter les totaux avec `GROUPING()`

Théorie : `Theorie_06_SQL.md`, section 5.

Objectif : remplacer les `NULL` de sous-total par des libellés lisibles.

```sql
SELECT
    CASE WHEN GROUPING(annee) = 1 THEN 'Toutes années'
         ELSE annee::text END AS annee,
    CASE WHEN GROUPING(pays) = 1 THEN 'Tous pays'
         ELSE pays END        AS pays,
    SUM(montant) AS ventes
FROM ventes
GROUP BY ROLLUP (annee, pays)
ORDER BY GROUPING(annee), annee, GROUPING(pays), pays;
```

Observez. Identique à l'entrepôt, à une seule chose près : la conversion en texte s'écrit `annee::text` au lieu de `CAST(... AS VARCHAR)`. Les deux écritures passent sur nos deux terrains, mais `::` est une commodité de PostgreSQL, que DuckDB a reprise, et non une écriture du langage (Cours 02, section 4). Quand vous écrivez du SQL destiné à voyager, préférez le `CAST`, qui est la forme standard.

Enlevez maintenant les deux `GROUPING()` de l'`ORDER BY`, pour ne garder que `ORDER BY annee, pays`, et relancez. La ligne « Tous pays » de chaque année quitte le bas de son bloc et va se ranger entre « Germany » et « United Kingdom » : elle est triée comme du texte, et le texte la range par son initiale. C'est la raison d'être du tri par `GROUPING()`, et elle apparaît seulement après qu'on a remplacé les `NULL` par des libellés. Tant que les sous-totaux restent des `NULL` bruts, un `ORDER BY annee, pays` suffit sur nos deux moteurs, qui rangent tous deux les `NULL` en fin de tri croissant.

2.3 — Agrégats conditionnels : un continent par colonne

Théorie : `Theorie_06_SQL.md`, section 6.

Objectif : les ventes par année, avec une colonne par continent.

```sql
SELECT annee,
       SUM(CASE WHEN continent = 'North America' THEN montant END) AS amerique_nord,
```


© 2026 iAlexMG. Tous droits réservés.

```sql
       SUM(CASE WHEN continent = 'Europe'        THEN montant END) AS europe,
       SUM(CASE WHEN continent = 'Pacific'       THEN montant END) AS pacifique
FROM ventes
GROUP BY annee
ORDER BY annee;
```

Observez. Rigoureusement la même requête que sur l'entrepôt. L'agrégat conditionnel est du SQL de base, il marche partout, et c'est précisément pour ça qu'il reste la valeur sûre.

PostgreSQL offre en plus une écriture réservée au standard, la clause `FILTER`, plus lisible dès qu'il y a plusieurs colonnes :

```sql
SELECT annee,
       SUM(montant) FILTER (WHERE continent = 'North America') AS amerique_nord,
       SUM(montant) FILTER (WHERE continent = 'Europe')        AS europe,
       SUM(montant) FILTER (WHERE continent = 'Pacific')       AS pacifique
FROM ventes
GROUP BY annee
ORDER BY annee;
```

Comparez les deux blocs à l'œil. Le second dit ce qu'il fait : somme le montant, en filtrant sur l'Europe. Le premier demande de comprendre que le `CASE` produit des `NULL` et que le `SUM` les ignore. C'est le même calcul, mais l'un s'explique et l'autre se déchiffre.

Piège — `FILTER` appartient à la norme SQL et passe sur nos deux terrains, entrepôt compris : la requête ci-dessus rend le même montant des deux côtés. Il n'est pourtant pas répandu partout, et notamment Snowflake ne le connaît pas, d'après sa documentation. Une requête écrite avec `FILTER` échouerait donc à une migration vers un moteur qui l'ignore, et il faudrait la réécrire en `SUM(CASE …)`. C'est exactement le genre de dette qu'on ne voit pas au moment où on l'accumule, et le fait qu'elle passe sur vos deux moteurs la rend plus discrète encore.

2.4 — Le tableau croisé, sans `PIVOT`

Théorie : `Theorie_06_SQL.md`, sections 7 et 9.

Objectif : obtenir le résultat de 2.3 avec une syntaxe dédiée, comme sur l'entrepôt, puis le replier comme le ferait `UNPIVOT`.

C'est ici que la divergence est complète : `PIVOT` n'existe pas dans PostgreSQL. Il n'y a pas d'équivalent direct, et ce n'est pas un oubli. Un `PIVOT` doit connaître la liste des colonnes de sortie au moment où la requête est analysée, or PostgreSQL exige que le type d'un résultat soit déterminé avant l'exécution. Un pivot dont les colonnes viendraient des données est donc, pour lui, une requête qui ne sait pas ce qu'elle renvoie.

Trois voies s'offrent à vous, dans l'ordre où je vous conseille de les envisager.

La première est celle de la section 2.3, l'agrégat conditionnel. C'est la bonne réponse dans la quasi-totalité des cas : on connaît presque toujours la liste des colonnes voulues, puisqu'un rapport a un format arrêté.

© 2026 iAlexMG. Tous droits réservés.

La deuxième est l'extension `tablefunc`, qui fournit une fonction `crosstab`. Installez-la et servez-vous-en une fois, parce qu'un défaut se démontre mieux qu'il ne se raconte :

```
CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM crosstab(
  $$SELECT annee, pays, ROUND(SUM(montant))
    FROM ventes
    WHERE pays IN ('Australia', 'Canada', 'France', 'Germany')
    GROUP BY annee, pays ORDER BY annee, pays$$
) AS ct(annee int, australie numeric, canada numeric, france numeric, allemagne numeric);
```

Observez. Le tableau croisé sort correctement, les mêmes chiffres qu'en 2.3. Deux détails à relever au passage : la requête source est passée en texte, entre `$$`, donc le moteur ne la vérifie qu'à l'exécution ; et la structure du résultat est déclarée à la main, dans le `AS ct(...)`, séparée de cette source. Maintenant, faites disparaître une seule valeur de la source, les ventes françaises de la première année, et relancez :

```
SELECT * FROM crosstab(
  $$SELECT annee, pays, ROUND(SUM(montant))
    FROM ventes
    WHERE pays IN ('Australia', 'Canada', 'France', 'Germany')
      AND NOT (annee = 2022 AND pays = 'France')
    GROUP BY annee, pays ORDER BY annee, pays$$
) AS ct(annee int, australie numeric, canada numeric, france numeric, allemagne numeric);
```

Observez, et regardez bien la première ligne. Les 246 861 $ de l'Allemagne se sont installés dans la colonne « france », et la colonne « allemagne » est vide. Aucune erreur, aucun avertissement. C'est le vrai piège de `crosstab` sous cette forme : il ne regarde pas le nom de la catégorie, il remplit les colonnes dans l'ordre où les lignes arrivent. Une catégorie absente d'un groupe décale tout ce qui la suit, et le rapport attribue des chiffres justes aux mauvais pays.

La parade est la forme à deux arguments, où l'on fournit la liste des catégories attendues :

```
SELECT * FROM crosstab(
  $$SELECT annee, pays, ROUND(SUM(montant))
    FROM ventes
    WHERE pays IN ('Australia', 'Canada', 'France', 'Germany')
      AND NOT (annee = 2022 AND pays = 'France')
    GROUP BY annee, pays ORDER BY annee, pays$$,
  $$VALUES ('Australia'), ('Canada'), ('France'), ('Germany')$$
) AS ct(annee int, australie numeric, canada numeric, france numeric, allemagne numeric);
```

Observez. Chaque valeur retrouve sa colonne, et la case sans donnée reste vide. Retenez la règle : si vous vous servez de `crosstab`, servez-vous en toujours à deux arguments. Cela dit, vous avez maintenant trois listes à tenir d'accord, celle du `WHERE`, celle des catégories et celle du `AS ct(...)`, ce qui est deux de trop pour un rapport qui vit longtemps. Je ne recommande `crosstab` que pour un besoin ponctuel.

© 2026 iAlexMG. Tous droits réservés.

La troisième voie est de renoncer au pivot en SQL et de le laisser à l'outil qui affiche. Un tableau croisé dans Power BI, dans un tableur ou dans une bibliothèque de graphiques se construit à partir de données au format long, une ligne par année et par continent, et c'est un format que le SQL produit sans effort :

```sql
SELECT annee, continent, SUM(montant) AS ventes
FROM ventes
GROUP BY annee, continent
ORDER BY annee, continent;
```

Observez ce dernier résultat, et retenez-le. C'est le format long, celui que `UNPIVOT` fabrique sur l'entrepôt à partir d'un tableau large. Il est plus long à l'écran, mais il est le seul qui reste juste quand un quatrième continent apparaît dans les données. Un pivot écrit en dur, lui, ignorera silencieusement la nouveauté.

Reste le cas où le tableau large existe déjà, et où c'est lui qu'il faut replier : une table reçue d'ailleurs, une colonne par année, qu'on veut ramener à deux colonnes. C'est le travail d'`UNPIVOT` sur l'entrepôt, et PostgreSQL le fait sans mot-clé dédié, avec une jointure latérale sur une liste de valeurs :

```sql
CREATE TEMP TABLE large AS
SELECT annee,
       ROUND(SUM(montant) FILTER (WHERE pays = 'Australia')) AS australie,
       ROUND(SUM(montant) FILTER (WHERE pays = 'Canada'))    AS canada,
       ROUND(SUM(montant) FILTER (WHERE pays = 'France'
                                  AND annee > 2022))       AS france
FROM ventes
GROUP BY annee;

SELECT l.annee, v.pays, v.ventes
FROM large AS l
CROSS JOIN LATERAL (VALUES ('Australia', l.australie),
                           ('Canada',    l.canada),
                           ('France',    l.france)) AS v(pays, ventes)
ORDER BY l.annee, v.pays;
```

Observez. Douze lignes, trois par année : la liste du `VALUES` fabrique une ligne par colonne repliée, et `LATERAL` lui donne accès à la ligne en cours. La case vide qu'on a fabriquée pour la France en 2022 sort ici comme une ligne à `ventes` vide, et non comme une ligne absente. C'est l'écart à connaître avec l'entrepôt, où `UNPIVOT` écarte les valeurs nulles par défaut : ajoutez `WHERE v.ventes IS NOT NULL` si vous voulez le même résultat des deux côtés. Des deux comportements, celui d'ici est le plus prudent : il ne fait rien disparaître sans qu'on l'ait demandé.

2.5 — Les raisons d'achat d'une commande

Théorie : `Theorie_06_SQL.md`, section 8.

Objectif : recoller les raisons d'achat de chaque commande, puis compter les combinaisons.

Le transactionnel garde ce que l'entrepôt n'a pas : pourquoi le client a acheté. Une commande peut avoir plusieurs raisons, rangées dans une table de liaison, `sales.salesorderheadersalesreason`, une ligne par couple commande et raison.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT hr.salesorderid,
       STRING_AGG(r.name, ', ' ORDER BY r.name) AS raisons,
       ARRAY_AGG(r.name ORDER BY r.name)        AS tableau,
       CARDINALITY(ARRAY_AGG(r.name))           AS nb
FROM sales.salesorderheadersalesreason AS hr
JOIN sales.salesreason                 AS r ON r.salesreasonid = hr.salesreasonid
GROUP BY hr.salesorderid
ORDER BY nb DESC, hr.salesorderid
LIMIT 3;
```

Observez. Les commandes 51214, 51298 et 51303 ont trois raisons chacune : On Promotion, Other, Price. La même liste sort en texte et en tableau, `{"On Promotion",Other,Price}`, et `CARDINALITY` compte les éléments du tableau. Recollée et triée, la liste devient une valeur qu'on peut regrouper :

```sql
SELECT raisons, COUNT(*) AS commandes
FROM (SELECT hr.salesorderid, STRING_AGG(r.name, ', ' ORDER BY r.name) AS raisons
      FROM sales.salesorderheadersalesreason AS hr
      JOIN sales.salesreason                 AS r ON r.salesreasonid = hr.salesreasonid
      GROUP BY hr.salesorderid) AS par_commande
GROUP BY raisons
ORDER BY commandes DESC
LIMIT 6;
```

Observez. Le prix seul l'emporte, 14 655 commandes, devant On Promotion (1 815), Manufacturer et Quality ensemble (1 551), On Promotion et Price (1 536), Review (1 245), Other et Price (1 129). Retirez le `ORDER BY r.name` de l'agrégat, et rien ne garantit plus que « Price, On Promotion » et « On Promotion, Price » se rangent dans le même groupe : c'est le tri qui fait d'une liste une valeur comparable.

Erreur voulue : l'écriture de la norme.

```sql
SELECT LISTAGG(name, ', ')
FROM production.productsubcategory
WHERE productcategoryid = 1;
```

Observez. Erreur : « la fonction listagg("Name", unknown) n'existe pas ». PostgreSQL n'a pas `LISTAGG`, que la norme définit et que l'entrepôt accepte ; `STRING_AGG` est la forme qui passe des deux côtés.

Partie 3 — Ce que le contraste vous a montré

Théorie : `Theorie_06_SQL.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

Ce qui a changé	Voie OLAP (entrepôt)	Voie OLTP (transactionnel)
L'année d'une vente	`CalendarYear`, une vraie colonne	`EXTRACT` sur `orderdate`, à nommer avant de regrouper dessus
Le nom d'un pays	en toutes lettres dans la dimension	un code sur deux lettres, à joindre pour l'afficher
Le nom de la colonne continent	`SalesTerritoryGroup`	`group`, un mot réservé, à écrire `t."group"`
`ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`	présents	présents, à l'identique
L'agrégat conditionnel	`SUM(CASE …)`, ou la clause `FILTER`	les deux aussi
Une ligne par groupe	`DISTINCT ON`, ou `QUALIFY ROW_NUMBER() … = 1`	`DISTINCT ON`
Le tableau croisé	`PIVOT` et `UNPIVOT`	aucun équivalent : agrégat conditionnel, `crosstab`, ou format long
Replier un tableau large	`UNPIVOT`, qui écarte les valeurs nulles	`CROSS JOIN LATERAL (VALUES …)`, qui les garde
Agréger en texte ou en liste	`STRING_AGG`, `LIST`, `ARRAY_AGG` et `LISTAGG`	`STRING_AGG` et `ARRAY_AGG`, sans `LISTAGG`

Deux leçons à emporter. La première est que les extensions du `GROUP BY` sont le morceau le plus portable de toute la formation : c'est du standard bien implanté, et vous pouvez les écrire sans réfléchir au moteur. Trois écritures de confort les accompagnent, `FILTER`, `DISTINCT ON` et `STRING_AGG`, que l'on croit souvent propres à PostgreSQL et que l'entrepôt accepte aussi. La seconde leçon est que le pivot, lui, est le morceau le moins portable de la formation, le seul écart de syntaxe que ce cours laisse debout, et que la parade n'est pas de chercher l'équivalent partout, mais de se demander si le pivot a sa place dans la requête. Livrer des données au format long et laisser l'outil d'affichage croiser, c'est une décision d'architecture qui vous épargne cette divergence pour de bon.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_06_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous savez produire des rapports complets des deux côtés, et vous avez vu où passe la frontière de la portabilité. Le Cours 07 change d'angle : les sous-requêtes corrélées et les opérateurs d'ensembles (`EXISTS`, `IN`, `UNION`, `INTERSECT`, `EXCEPT`), pour poser des questions que l'agrégation seule ne sait pas exprimer.

© 2026 iAlexMG. Tous droits réservés.